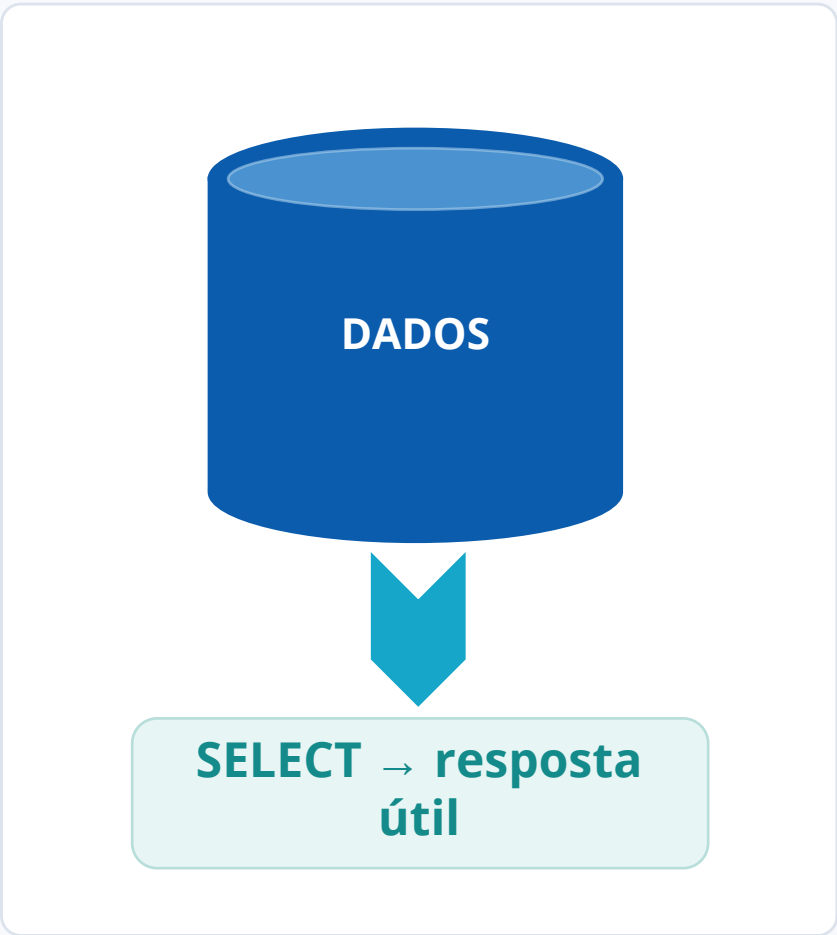
Consultas SQL: da tabela à resposta

Tópico: SELECT, projeção, seleção, ordenação e condições

- Consultar um banco significa formular uma pergunta aos dados e receber um conjunto de linhas e colunas como resposta.
- O comando SELECT é a base das consultas SQL e permite escolher colunas, filtrar registros e definir a ordem de apresentação.
- Nesta aula usaremos uma tabela de alunos para construir consultas gradualmente, sempre relacionando a sintaxe ao significado da operação.
- O objetivo não é decorar comandos, mas aprender a transformar uma necessidade de informação em uma consulta clara e verificável.



Objetivos de aprendizagem

Tópico: O que o aluno deverá conseguir fazer

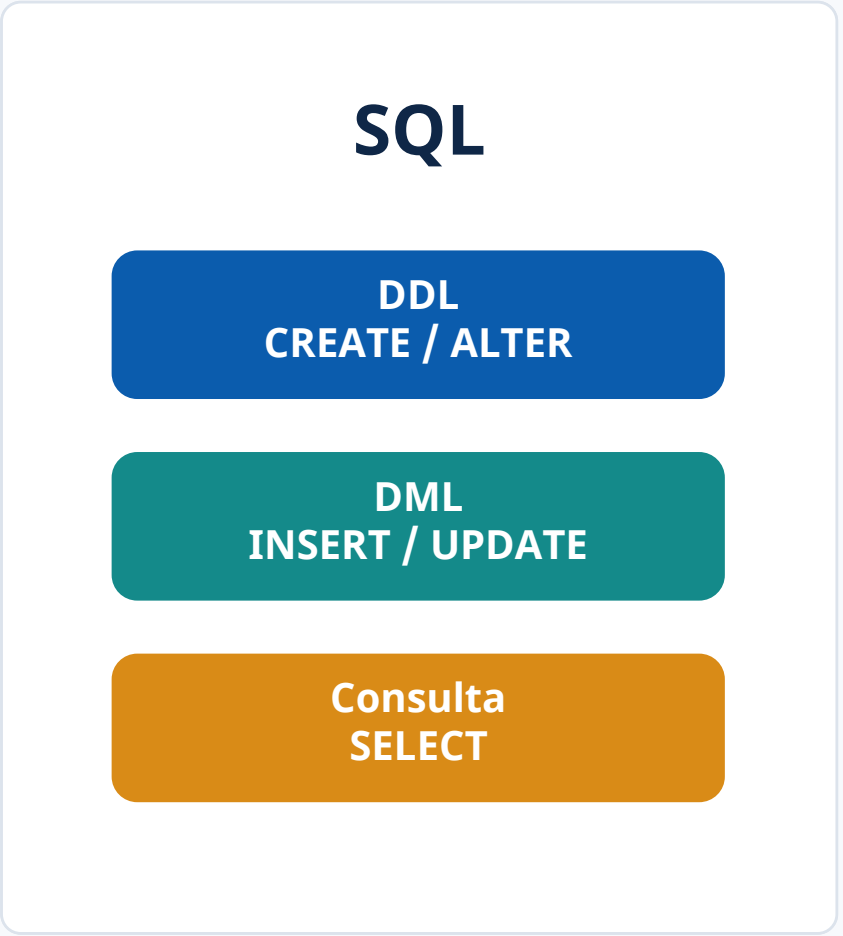
- Identificar a estrutura de uma consulta SELECT e diferenciar a lista de colunas, a tabela de origem e as cláusulas opcionais.
- Aplicar projeção para escolher atributos e seleção para filtrar linhas conforme condições definidas com operadores comparativos e lógicos.
- Ordenar resultados com ORDER BY, incluindo critérios ascendentes, descendentes e combinações de mais de uma coluna.
- Executar consultas em uma ferramenta gratuita, interpretar resultados e corrigir erros comuns de sintaxe ou de lógica.



Onde o SELECT se encaixa na SQL?

Tópico: Consulta dentro da linguagem SQL

- SQL reúne comandos para definir estruturas, manipular dados, controlar acesso e consultar informações armazenadas em bancos relacionais.
- CREATE, ALTER e DROP pertencem ao conjunto usado para definir estruturas; INSERT, UPDATE e DELETE alteram os dados armazenados.
- SELECT é usado para consultar dados e, em materiais introdutórios, costuma aparecer



Cenário didático: tabela aluno

Tópico: Uma mesma base para todos os exemplos

- Usaremos a tabela aluno com dados acadêmicos simples: identificador, nome, curso, período, cidade, idade, média e situação de matrícula.
- Cada linha representa um aluno; cada coluna representa uma característica desse aluno, seguindo a organização típica do modelo relacional.
- Os exemplos foram escolhidos para permitir filtros numéricos, filtros de texto, combinações lógicas, valores nulos e diferentes ordenações.
- Manter a mesma tabela ao longo da aula facilita comparar como cada nova cláusula modifica apenas uma parte da consulta.

Tabela aluno

id	nome	curso	média
1	Ana	SI	8,5
2	Bruno	ADS	6,8
3	Carla	SI	9,1
4	Diego	CC	5,9
5	Elisa	ADS	7,7
6	Felipe	SI	NULL

Estrutura básica de uma consulta

Tópico: SELECT ... FROM ...

- SELECT indica quais valores devem aparecer no resultado; FROM informa a tabela que fornece os dados usados pela consulta.
- A lista após SELECT pode conter uma ou várias colunas, expressões calculadas e, em consultas mais avançadas, funções e agregações.
- As palavras-chave SQL são tradicionalmente escritas em maiúsculas por legibilidade, embora muitos SGBDs aceitem também letras minúsculas.
- O ponto e vírgula encerra a instrução e é uma boa prática, especialmente quando vários comandos são executados no mesmo editor.

Estrutura mínima

```
SELECT nome, curso
FROM aluno;
```

SELECT *: consultar todas as colunas

Tópico: O asterisco como atalho

- O asterisco representa todas as colunas disponíveis na tabela e é útil para explorar rapidamente uma base pequena durante aprendizagem ou diagnóstico.
- Em sistemas reais, SELECT * pode trazer dados desnecessários, aumentar tráfego e tornar o código mais sensível a mudanças na estrutura da tabela.
- Quando sabemos quais informações serão usadas, é preferível listar explicitamente as colunas necessárias no resultado da consulta.
- A diferença é importante: consultar todos os registros não significa obrigatoriamente consultar todas as colunas, e vice-versa.

Consulta → resultado

```
SELECT *
FROM aluno;
```

id	nome	curso
1	Ana	SI
2	Bruno	ADS
3	Carla	SI

Projeção: escolher colunas

Tópico: Reduzindo a largura do resultado

- Projeção é a operação conceitual de escolher quais atributos ou colunas participarão do resultado, descartando as demais informações da tabela.
- Em SQL, a lista escrita após SELECT desempenha esse papel: SELECT nome, curso retorna somente essas duas colunas para cada linha considerada.
- A projeção melhora a clareza porque o resultado contém apenas os dados necessários para responder à pergunta proposta.
- Exemplo de requisito: “mostrar apenas o nome e a média dos alunos” é traduzido diretamente pela escolha dessas colunas.

id	nome	curso	média
1	Ana	SI	8,5
2	Bruno	ADS	6,8
3	Carla	SI	9,1



nome	média
Ana	8,5
Bruno	6,8

Projeção e DISTINCT

Tópico: Duplicatas no resultado SQL

- Uma diferença importante é que uma projeção conceitual da álgebra relacional trabalha com conjuntos, enquanto resultados SQL podem manter valores repetidos.
- SELECT curso FROM aluno pode mostrar “SI” várias vezes porque vários alunos pertencem ao mesmo curso.
- A palavra DISTINCT solicita a remoção de linhas duplicadas do resultado produzido pela lista de colunas selecionadas.
- Use DISTINCT quando a pergunta realmente exigir valores únicos; aplicá-lo sem necessidade pode esconder informações relevantes ou adicionar processamento.

Consulta → resultado

```
SELECT DISTINCT curso
FROM aluno;
```

curso
SI
ADS
CC

Aliases com AS

Tópico: Renomeando colunas no resultado

- Um alias é um nome temporário atribuído a uma coluna ou expressão apenas para tornar o resultado mais legível para quem o consulta.
- A palavra AS torna a intenção explícita: SELECT nome AS aluno, media AS nota_final altera os rótulos exibidos, sem renomear a tabela original.
- Aliases são especialmente úteis quando usamos expressões calculadas ou nomes técnicos pouco amigáveis para relatórios e aplicações.
- Alguns SGBDs permitem omitir AS, mas usá-lo em exemplos introdutórios deixa a leitura mais clara e reduz ambiguidades.

Consulta → resultado

```
SELECT nome AS aluno,
       media AS nota_final
FROM aluno;
```

aluno	nota_final
Ana	8,5
Bruno	6,8

Expressões no SELECT

Tópico: Calculando valores durante a consulta

- A lista de projeção não precisa conter somente nomes de colunas: ela também pode incluir cálculos construídos a partir dos valores de cada linha.
- Por exemplo, media * 10 pode transformar uma média de 0 a 10 em uma pontuação de 0 a 100 apenas no resultado.
- O banco calcula a expressão para cada registro retornado e pode apresentar o valor com um alias descritivo.
- Esse recurso permite preparar informações para relatórios sem alterar os valores armazenados originalmente na tabela.

Consulta → resultado

```
SELECT nome,
       media * 10 AS pontos
FROM aluno;
```

nome	pontos
Ana	85
Carla	91

Seleção com WHERE

Tópico: Filtrando linhas

- WHERE define uma condição que cada linha deve satisfazer para permanecer no resultado; as linhas que não atendem ao critério são descartadas.
- Essa operação corresponde ao conceito de seleção de linhas: em vez de reduzir colunas, reduzimos os registros analisados pelo usuário.
- A condição pode comparar números, textos, datas ou outras expressões, dependendo dos tipos de dados existentes na tabela.
- Exemplo: WHERE curso = 'SI' mantém somente alunos cujo valor da coluna curso corresponde ao texto informado.

Consulta → resultado

```
SELECT nome, curso
FROM aluno
WHERE curso = 'SI';
```

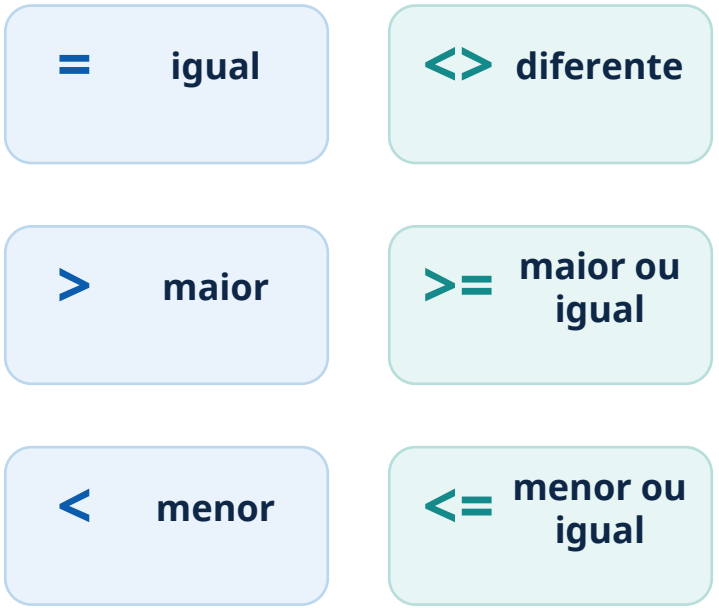
nome	curso
Ana	SI
Carla	SI
Felipe	SI

Documentação PostgreSQL — consultas e operadores

Operadores comparativos

Tópico: Transformando critérios em condições

- Operadores comparativos confrontam dois valores e produzem uma condição lógica: verdadeira, falsa ou, em situações com NULL, desconhecida.
- Os principais são =, <>, >, >=, < e <=; muitos SGBDs também aceitam != como alternativa para “diferente”.
- Escolher o operador correto depende da pergunta: media >= 7 representa “média maior ou igual a sete”, incluindo exatamente o valor sete.
- Comparações podem ser combinadas com operadores lógicos para representar regras mais próximas das necessidades reais de um sistema.

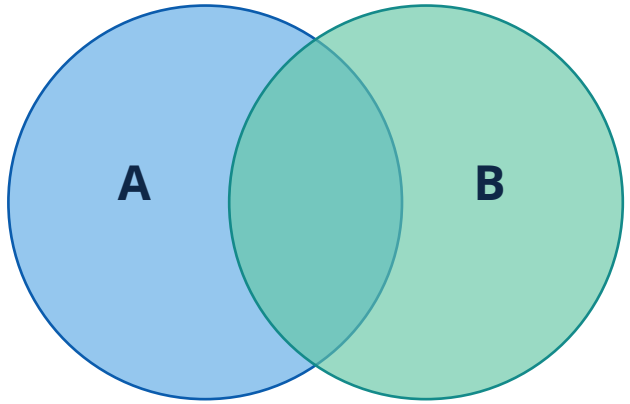


Documentação PostgreSQL — consultas e operadores

Operador lógico AND

Tópico: Todas as condições devem ser verdadeiras

- AND combina condições e exige que todas sejam satisfeitas para que a linha seja incluída no resultado final da consulta.
- WHERE curso = 'SI' AND media >= 7 seleciona somente alunos de Sistemas de Informação que também possuem média igual ou superior a sete.
- Se uma das condições for falsa, a linha não atende ao filtro completo e não aparece na resposta.
- AND é apropriado quando o requisito usa expressões como “e”, “ao mesmo tempo”, “também” ou “além disso”.

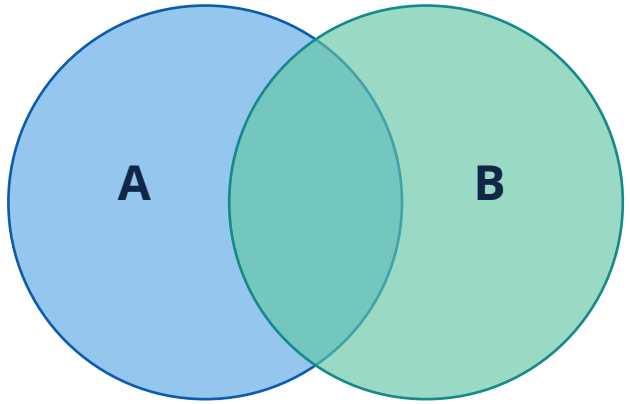


AND: somente a interseção precisa ser verdadeira.

Operador lógico OR

Tópico: Basta uma condição ser verdadeira

- OR combina alternativas: a linha é mantida quando pelo menos uma das condições avaliadas é verdadeira, podendo inclusive satisfazer ambas.
- WHERE cidade = 'Vitória' OR cidade = 'Vila Velha' retorna alunos de qualquer uma das duas cidades indicadas.
- OR costuma representar requisitos escritos com “ou”, “qualquer um destes valores” ou “uma condição ou outra”.
- Quando muitas comparações de igualdade são feitas sobre a mesma coluna, o operador IN pode tornar a consulta mais curta e legível.

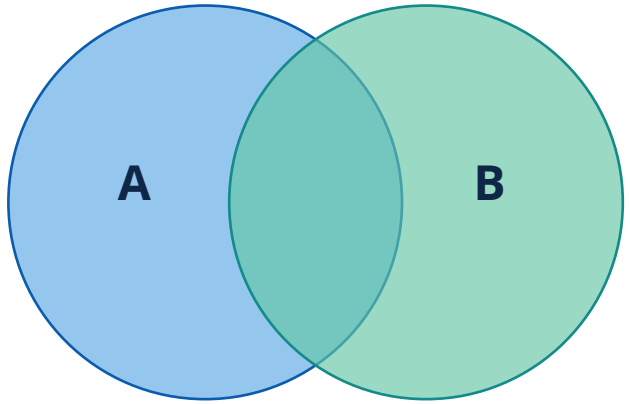


OR: A, B ou ambos podem ser verdadeiros.

Operador lógico NOT

Tópico: Negando uma condição

- NOT inverte o resultado lógico de uma condição e é útil para expressar requisitos de exclusão, como “não pertence”, “não está” ou “não corresponde”.
- WHERE NOT curso = 'SI' mantém os registros cuja condição curso = 'SI' não é verdadeira, observando o tratamento especial de valores NULL.
- Também existem formas como NOT IN, NOT BETWEEN e NOT LIKE, que tornam algumas negações mais naturais de ler.
- Em consultas complexas, parênteses ajudam a deixar claro exatamente qual parte da condição está sendo negada.



NOT: inverte a condição selecionada.

Precedência: AND, OR e parênteses

Tópico: A ordem muda o resultado

- Quando uma condição mistura operadores lógicos, o SGBD segue regras de precedência; em geral, NOT é avaliado antes de AND, e AND antes de OR.
- Por isso, uma expressão sem parênteses pode produzir um conjunto de linhas diferente daquele imaginado pelo programador.
- Parênteses tornam a intenção explícita e devem ser usados quando a regra de negócio combina alternativas com requisitos obrigatórios.
- Compare: curso = 'SI' OR curso = 'ADS' AND media >= 7 com (curso = 'SI' OR curso = 'ADS') AND media >= 7.

Parênteses deixam a regra clara

```
-- Sem agrupamento
curso = 'SI' OR
curso = 'ADS' AND media >= 7

-- Intenção explícita
(curso = 'SI' OR curso = 'ADS')
AND media >= 7
```

Documentação SQLite — SELECT e expressões

BETWEEN: filtrando intervalos

Tópico: Limites inclusivos

- BETWEEN expressa um intervalo de forma compacta e, na forma usual, inclui os valores das duas extremidades informadas na condição.
- WHERE idade BETWEEN 18 AND 21 equivale conceitualmente a idade >= 18 AND idade <= 21.
- Esse operador é útil para faixas numéricas e também pode ser aplicado a datas, respeitando os tipos e formatos do SGBD utilizado.
- Para excluir o intervalo, podemos usar NOT BETWEEN, sempre verificando se a regra de negócio considera os limites como incluídos.

Consulta → resultado

```
SELECT nome, idade
FROM aluno
WHERE idade BETWEEN 18 AND 21;
```

nome	idade
Ana	19
Carla	20
Diego	18
Felipe	21

Documentação PostgreSQL — consultas e operadores

IN: comparando com uma lista

Tópico: Múltiplos valores possíveis

- IN verifica se um valor pertence a uma lista de alternativas e evita repetir várias condições de igualdade conectadas por OR.
- WHERE cidade IN ('Vitória', 'Serra', 'Vila Velha') seleciona registros cuja cidade corresponde a qualquer item informado na lista.
- A forma equivalente com OR seria mais longa e mais difícil de manter quando a quantidade de opções aumenta.
- NOT IN faz a operação inversa, mas exige atenção adicional quando listas ou subconsultas podem envolver valores NULL.

Lista de alternativas

```
SELECT nome, cidade
FROM aluno
WHERE cidade IN (
  'Vitória',
  'Serra',
  'Vila Velha'
);
```

Documentação SQLite — SELECT e expressões

LIKE: buscando padrões em textos

Tópico: Curingas % e _

- LIKE compara textos usando padrões; o símbolo % representa uma sequência de zero ou mais caracteres, enquanto _ representa um único caractere.
- WHERE nome LIKE 'A%' procura nomes iniciados por A; WHERE nome LIKE '_na%' exige um caractere antes de “na”.
- O comportamento de maiúsculas, minúsculas e acentos pode variar conforme o SGBD, a configuração e a regra de collation utilizada.
- LIKE é útil para buscas simples, mas não deve substituir mecanismos específicos quando a aplicação exige pesquisa textual avançada.

Padrões de texto

```
-- começa com A
WHERE nome LIKE 'A%'

-- termina com a
WHERE nome LIKE '%a'

-- um caractere + 'na'
WHERE nome LIKE '_na%'
```

Documentação SQLite — SELECT e expressões

NULL: ausência de valor conhecido

Tópico: Use IS NULL, não = NULL

- NULL representa ausência de um valor conhecido; ele não significa zero, texto vazio ou automaticamente “não se aplica”.
- Comparações comuns com NULL não funcionam como uma igualdade normal, pois o resultado pode ser desconhecido em vez de verdadeiro ou falso.
- Para localizar valores ausentes, use IS NULL; para exigir um valor presente, use IS NOT NULL.
- No exemplo, WHERE media IS NULL identifica alunos cuja média ainda não foi registrada, sem confundir essa situação com média igual a zero.

Consulta → resultado

```
SELECT nome, media
FROM aluno
WHERE media IS NULL;
```

nome	média
Felipe	NULL

Documentação PostgreSQL — consultas e operadores

ORDER BY: ordenando o resultado

Tópico: ASC e DESC

- ORDER BY define a ordem em que as linhas serão apresentadas ao usuário, sem alterar a ordem física em que os dados estão armazenados.
- ASC indica ordem crescente e é normalmente o padrão; DESC indica ordem decrescente, útil para mostrar maiores valores primeiro.
- SELECT nome, media FROM aluno ORDER BY media DESC organiza os alunos da maior para a menor média.
- Sem ORDER BY, não devemos depender da ordem observada em uma execução, pois o SGBD pode devolver linhas em ordem não especificada.

Consulta → resultado

```
SELECT nome, media
FROM aluno
ORDER BY media DESC;
```

nome	média
Carla	9,1
Ana	8,5
Elisa	7,7
Bruno	6,8

Documentação PostgreSQL — consultas e operadores

Ordenação por várias colunas

Tópico: Critério principal e desempate

- ORDER BY aceita vários critérios separados por vírgulas; o primeiro é usado como ordenação principal e os seguintes resolvem empates.
- ORDER BY curso ASC, media DESC primeiro agrupa alfabeticamente os cursos e, dentro de cada curso, coloca as maiores médias antes.
- Cada coluna pode ter sua própria direção de ordenação, permitindo combinações como curso ASC e nome DESC.
- Esse recurso é comum em relatórios, listas de classificação e telas que precisam de resultados previsíveis para o usuário.

Critério + desempate

```
SELECT nome, curso, media
FROM aluno
ORDER BY curso ASC,
         media DESC;
```

Combinando projeção, seleção e ordenação

Tópico: Uma consulta completa e legível

- Consultas reais combinam as ideias anteriores: escolhem as colunas necessárias, filtram as linhas relevantes e ordenam o conjunto resultante.
- Leia a consulta em partes: SELECT define a saída, FROM define a origem, WHERE restringe os registros e ORDER BY organiza a apresentação.
- Quebrar a instrução em linhas melhora a leitura e facilita localizar erros, embora espaços e quebras normalmente não alterem o significado SQL.
- Construir a consulta gradualmente é uma estratégia segura: execute primeiro o SELECT, depois acrescente o filtro e, por fim, a ordenação.

Consulta completa

```
SELECT nome, curso, media
FROM aluno
WHERE ativo = 1
  AND media >= 7
ORDER BY media DESC;
```

Ordem escrita x ordem lógica

Tópico: Como pensar no processamento da consulta

- Embora escrevamos SELECT antes de FROM e WHERE, uma forma didática de entender a execução lógica é começar pela origem dos dados.
- Primeiro o SGBD determina o conjunto indicado por FROM; depois aplica o filtro de WHERE; em seguida calcula a lista de SELECT.
- Após a produção do resultado, ORDER BY pode organizar as linhas conforme os critérios definidos pelo programador.
- Entender essa ordem ajuda a explicar por que alguns nomes ou aliases ainda não estão disponíveis em determinadas partes de consultas mais complexas.



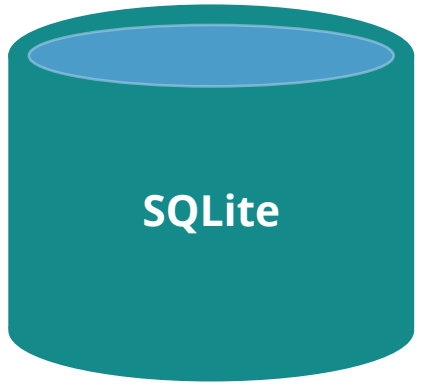
Modelo mental da execução lógica

Documentação SQLite — SELECT e expressões

Prática: ambiente gratuito

Tópico: DB Browser for SQLite

- DB Browser for SQLite é uma ferramenta visual e de código aberto que permite criar bancos SQLite, editar registros e executar consultas SQL.
- Baixe a versão oficial, instale ou utilize o pacote adequado ao sistema operacional e crie um novo arquivo chamado faculdade.db.
- Abra a guia “Execute SQL”, cole os comandos do exercício e execute cada consulta separadamente para observar imediatamente o resultado.
- Para esta aula, SQLite é suficiente porque não exige configurar um servidor e mantém o banco inteiro em um arquivo local.



DB Browser for SQLite

Criar • consultar • visualizar

DB Browser for SQLite — site oficial

Prática: criar a base de exemplo

Tópico: Tabela e registros para a demonstração

- Na guia de execução SQL, crie a tabela aluno com colunas simples e insira os registros de exemplo mostrados durante a aula.
- Execute o bloco de criação apenas uma vez; se repetir CREATE TABLE sem remover a tabela, o SGBD informará que o objeto já existe.
- Depois dos INSERTs, confirme os dados com SELECT * FROM aluno; e compare o resultado com a tabela apresentada nos slides.
- Mantenha um registro com média NULL para praticar IS NULL e perceber a diferença entre valor ausente e valor numérico zero.

Base mínima para a prática

```
CREATE TABLE aluno (
 id INTEGER PRIMARY KEY,
 nome TEXT NOT NULL,
 curso TEXT NOT NULL,
 cidade TEXT, idade INTEGER,
 media REAL, ativo INTEGER
);

INSERT INTO aluno VALUES
(1,'Ana','SI','Vitória',19,8.5,1);
```

DB Browser for SQLite — site oficial

Prática guiada: quatro consultas

Tópico: Do requisito em português para SQL

- Consulta 1: mostre nome e curso de todos os alunos, praticando projeção sem filtro.
- Consulta 2: encontre alunos ativos com média maior ou igual a sete, combinando comparação e AND.
- Consulta 3: liste alunos de Vitória ou Vila Velha em ordem alfabética de nome, combinando filtro e ORDER BY.
- Consulta 4: identifique registros sem média cadastrada; antes de executar, peça aos alunos que prevejam quantas linhas serão retornadas.

1. Projeção

2. AND + comparação

3. IN + ORDER BY

4. IS NULL

SQLBolt — exercícios interativos

Erros comuns em consultas iniciais

Tópico: Sintaxe correta não garante lógica correta

- Esquecer aspas simples em textos, como cidade = Vitória, faz o SGBD interpretar Vitória como identificador em vez de valor textual.
- Usar = NULL em vez de IS NULL impede o filtro esperado, porque valores nulos seguem regras próprias de comparação.
- Misturar AND e OR sem parênteses pode retornar linhas diferentes da regra pretendida, mesmo que a consulta seja sintaticamente válida.
- Confiar na ordem visual sem ORDER BY ou selecionar colunas desnecessárias com * são hábitos que prejudicam consultas previsíveis e claras.

✗ cidade = Vitória

✓ cidade = 'Vitória'

✗ media = NULL

✓ media IS NULL

✗ sem ORDER BY

✓ ORDER BY ...

Desafio em sala

Tópico: Construa a consulta por etapas

- Requisito: “mostrar nome, curso e média dos alunos ativos de SI ou ADS, com média entre 7 e 10, ordenados da maior para a menor média”.
- Primeiro identifique as colunas da projeção; depois transforme cada parte do requisito em condições e escolha os operadores adequados.
- Use parênteses para agrupar a alternativa entre cursos e combine esse grupo com as demais exigências usando AND.
- Antes de executar, leia a consulta em voz alta e verifique se a lógica SQL corresponde exatamente à frase do requisito.

Uma solução possível

```
SELECT nome, curso, media
FROM aluno
WHERE ativo = 1
  AND curso IN ('SI','ADS')
  AND media BETWEEN 7 AND
10
ORDER BY media DESC;
```

SQLBolt — exercícios interativos

Avaliação e referências

Tópico: Verificação da aprendizagem e fontes

- Avaliação: escreva uma consulta que retorne nome e média dos alunos de SI com média >= 7, ordenando da maior para a menor média.
- Avaliação: explique, em uma frase, a diferença entre projeção de colunas e seleção de linhas e dê um exemplo SQL de cada operação.
- Referências principais: PICHETTI; VIDA; CORTES, Banco de dados, 2021; documentação atual do PostgreSQL e do SQLite para SELECT, comparação e ordenação.
- Prática complementar: DB Browser for SQLite para execução local e SQLBolt para exercícios interativos de SELECT, filtros e ORDER BY.

Fontes para continuar

PostgreSQL — SELECT / ORDER BY

SQLite — SELECT / expressions

DB4S — prática local

SQLBolt — exercícios interativos

PostgreSQL • SQLite • DB Browser • SQLBolt